CUSTOMER SHOPPPING BEHAVIOR ANALYSIS

By EL IGGOUCH

PROBLEM STATEMENT :

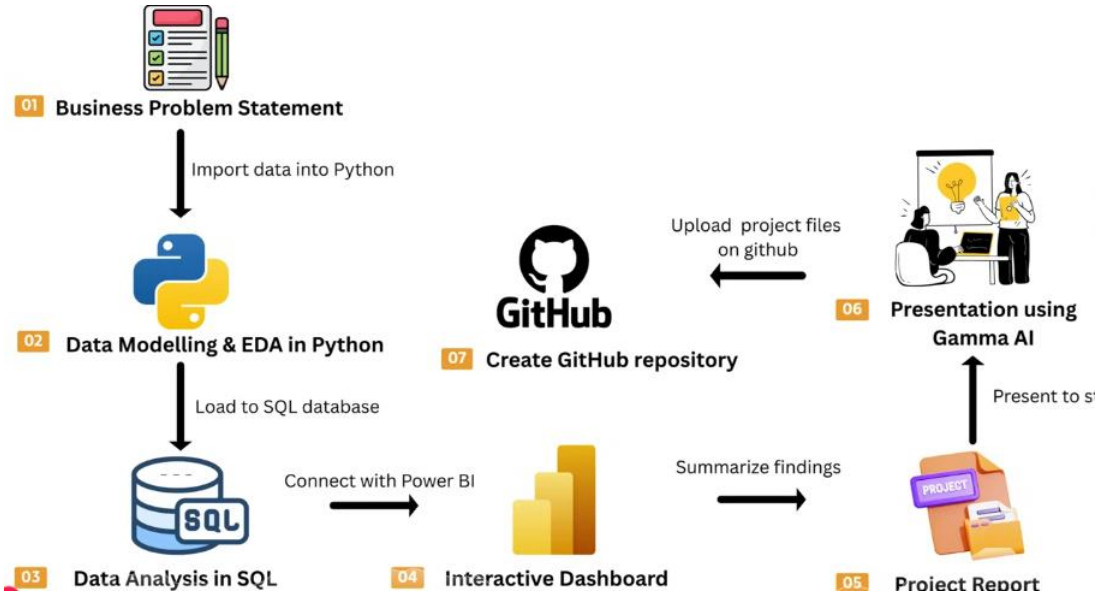
Problem Statement

A leading retail company wants to better understand its customers' shopping behavior in order to improve sales, customer satisfaction, and long-term loyalty. The management team has noticed changes in purchasing patterns across demographics, product categories, and sales channels (online vs. offline). They are particularly interested in uncovering which factors, such as discounts, reviews, seasons, or payment preferences, drive consumer decisions and repeat purchases.

You are tasked with analyzing the company's consumer behavior dataset to answer the following overarching business question:

"How can the company leverage consumer shopping data to identify trends, improve customer engagement, and optimize marketing and product strategies?"

Architecture we will use :



1 _ PROJECT OVERVIEW :

this project analyzes customer shopping behavior using transactional data from 3900 purchases across various product categories. The goal is to uncover insights into spending patterns,customer segments,product preferences and subscription behavior to guide strategic business decision.

2 _ DATAASET SUMMARY :

- Rows : 3900
- Columns : 18
- Key Features :
- Customer Demographics : (Age,Gender,Location,Subscription status)
- Purchase details : (item purchased ,categories , purchase amount ,season, size , color)

- Shopping behavior : (Discount Applied,Promo code Used , Previous Purchases , Frequency of Purchases , Review Rating , Shipping type)
- Missing Data : 37 values in Review Rating column

3 _ EXPLORATORY DATA ANALYSIS USING PYTHON :

3.1 _ EXTRACTING & LOADING :

We will make sure to extract the data from our CSV file using Pandas lab with the help

Path lib . we will copy the path of our csv file on our computer in past it on the

Read_csv file function with the help of pandas lib .

```
PROJECT_ROOT = Path.cwd().parent.parent.parent.parent.parent
print(PROJECT_ROOT)
DATASET_PATH = PROJECT_ROOT / "PROJECTS" / "RETAIL COMPANY PROJECT" / "DATASET" / "customer_shopping_behavior.csv"
print(DATASET_PATH)
```

```
dataset = pd.read_csv(DATASET_PATH)
df = dataset.copy()
df.head()
```

After that we will display the first 5 column to check if there is a bug or runtime error.

	Customer ID	Age	Gender	Item Purchased	Category	Purchase Amount (USD)	Location	Size	Color	Season	Review Rating	Subscription Status	Shipp T
0	1	55	Male	Blouse	Clothing	53	Kentucky	L	Gray	Winter	3.1	Yes	Exp
1	2	19	Male	Sweater	Clothing	64	Maine	L	Maroon	Winter	3.1	Yes	Exp
2	3	50	Male	Jeans	Clothing	73	Massachusetts	S	Maroon	Spring	3.1	Yes	F Shipp
3	4	21	Male	Sandals	Footwear	90	Rhode Island	M	Maroon	Spring	3.5	Yes	Next
4	5	45	Male	Blouse	Clothing	49	Oregon	M	Turquoise	Spring	2.7	Yes	F Shipp

So as we can see we have display the the first 5 column and our dataset is safly loaded in our pandas lib.

3.2 _ sample data exploration :

Now our dataset is ready for analysis but before that , first of all we need to perform some sample data validation and exploration to make sure that our dataset set is clean and reliable for analysis.

```
df.info()
✓ 0.0s
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3900 entries, 0 to 3899
Data columns (total 18 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   Customer ID             3900 non-null   int64
 1   Age                     3900 non-null   int64
 2   Gender                  3900 non-null   object
 3   Item Purchased          3900 non-null   object
 4   Category                3900 non-null   object
 5   Purchase Amount (USD)   3900 non-null   int64
 6   Location                3900 non-null   object
 7   Size                    3900 non-null   object
 8   Color                   3900 non-null   object
 9   Season                  3900 non-null   object
 10  Review Rating           3863 non-null   float64
 11  Subscription Status     3900 non-null   object
 12  Shipping Type           3900 non-null   object
 13  Discount Applied        3900 non-null   object
 14  Promo Code Used         3900 non-null   object
 15  Previous Purchases      3900 non-null   int64
 16  Payment Method          3900 non-null   object
 17  Frequency of Purchases  3900 non-null   object
dtypes: float64(1), int64(4), object(13)
memory usage: 548.6+ KB
```

The `info()` method was applied to the dataset to obtain a structural overview of the DataFrame. This function provides information about the number of observations, the list of variables, their corresponding data types, and the number of non-null values in each column. This step helps identify potential issues such as missing data and incorrect data types, which are important considerations before proceeding with data cleaning and analysis.

As we can see from the result :

We have 13 object type means strings , and 4 intiger datatypes and 1 float datatype

Also we have any null values in our dataframe , and we have some messing values on the review rating columns

now befor starting cleaning the data , I wanna make sure that the columns name dosent contains any spaces or upperlower letter problem so :

```
df.columns=df.columns.str.lower()
df.columns = df.columns.str.replace(" ","_")
df.rename(columns={"purchase_amount_(usd)":"purchase_amount"},inplace=True)
df.columns
```

✓ 0.0s

```
Index(['customer_id', 'age', 'gender', 'item_purchased', 'category',
       'purchase_amount', 'location', 'size', 'color', 'season',
       'review_rating', 'subscription_status', 'shipping_type',
       'discount_applied', 'promo_code_used', 'previous_purchases',
       'payment_method', 'frequency_of_purchases'],
      dtype='object')
```

The column names of the dataset were standardized to improve readability and ensure consistency during the analysis process. First, all column names were converted to lowercase to maintain a uniform naming convention. Next, spaces within column names were replaced with underscores to facilitate easier referencing in the analysis code. Additionally, the column `purchase_amount_(usd)` was renamed to `purchase_amount` to simplify its representation while preserving its meaning. Finally, the list of column names was displayed to confirm that the transformations were successfully applied.

And after that we will perform a simple descriptive stats :

```
df.describe(include="all").T
```

✓ 0.0s

	count	unique	top	freq	mean	std	min	25%	50%	75%	max
customer_id	3900.0	NaN	NaN	NaN	1950.5	1125.977353	1.0	975.75	1950.5	2925.25	3900.0
age	3900.0	NaN	NaN	NaN	44.068462	15.207589	18.0	31.0	44.0	57.0	70.0
gender	3900	2	Male	2652	NaN	NaN	NaN	NaN	NaN	NaN	NaN
item_purchased	3900	25	Blouse	171	NaN	NaN	NaN	NaN	NaN	NaN	NaN
category	3900	4	Clothing	1737	NaN	NaN	NaN	NaN	NaN	NaN	NaN
purchase_amount	3900.0	NaN	NaN	NaN	59.764359	23.685392	20.0	39.0	60.0	81.0	100.0
location	3900	50	Montana	96	NaN	NaN	NaN	NaN	NaN	NaN	NaN
size	3900	4	M	1755	NaN	NaN	NaN	NaN	NaN	NaN	NaN
color	3900	25	Olive	177	NaN	NaN	NaN	NaN	NaN	NaN	NaN
season	3900	4	Spring	999	NaN	NaN	NaN	NaN	NaN	NaN	NaN
review_rating	3863.0	NaN	NaN	NaN	3.750065	0.716983	2.5	3.1	3.8	4.4	5.0
subscription_status	3900	2	No	2847	NaN	NaN	NaN	NaN	NaN	NaN	NaN
shipping_type	3900	6	Free Shipping	675	NaN	NaN	NaN	NaN	NaN	NaN	NaN
discount_applied	3900	2	No	2223	NaN	NaN	NaN	NaN	NaN	NaN	NaN
promo_code_used	3900	2	No	2223	NaN	NaN	NaN	NaN	NaN	NaN	NaN
previous_purchases	3900.0	NaN	NaN	NaN	25.351538	14.447125	1.0	13.0	25.0	38.0	50.0
payment_method	3900	6	PayPal	677	NaN	NaN	NaN	NaN	NaN	NaN	NaN

A descriptive statistical summary of the dataset was generated using the `describe()` function. By specifying `include="all"`, the summary statistics were calculated for both numerical and categorical variables. The output provides key information such as counts, measures of central tendency, dispersion statistics, and frequency information for categorical variables. The result was transposed using `.T` to

present each variable as a row, which improves readability and facilitates easier interpretation of the dataset's characteristics.

Based on the descriptive statistical summary, most numerical variables do not show strong indications of outliers. This observation is supported by the proximity between the mean and the median (50th percentile) values across many variables. Such similarity between these two measures of central tendency typically suggests a relatively balanced distribution with limited influence from extreme observations.

```
df.isnull().sum()
✓ 0.0s

customer_id               0
age                       0
gender                    0
item_purchased            0
category                  0
purchase_amount           0
location                  0
size                      0
color                     0
season                    0
review_rating            37
subscription_status       0
shipping_type             0
discount_applied          0
promo_code_used           0
previous_purchases        0
payment_method            0
frequency_of_purchases    0
dtype: int64
```

The presence of missing values in the dataset was examined using the `isnull().sum()` method. This operation identifies null values in each column and calculates the total number of missing observations per variable. Detecting missing data is an important step in the data preprocessing phase, as it allows the analyst to evaluate the completeness of the dataset and determine whether further data cleaning or imputation procedures are required before proceeding with the analysis.

As shown in the previous step, the review_rating column contains 37 missing values**.** Handling missing data is an important part of the data preprocessing process because incomplete data can negatively affect the reliability of the analysis results. There are generally two possible approaches to address this issue: removing the column **or** imputing the missing values**.** Removing the column may be appropriate if the variable is not important for the analysis or if it contains a large proportion of missing data.

In cases where the variable is relevant to the analysis, a more appropriate solution is data imputation**,** which consists of replacing the missing values with estimated values. For numerical variables, simple imputation techniques include replacing missing values with **the** mean **or** median of the column. These methods are commonly used when the data follows a relatively normal or symmetric distribution.

In addition to these basic methods, more advanced imputation techniques can also be applied. These **include** machine learning–based approaches **such as** K-Nearest Neighbors (KNN) imputation, regression-based imputation, or ensemble learning methods**.** These techniques estimate missing values by learning patterns from other variables in the dataset, which can sometimes produce more accurate imputations when the dataset contains complex relationships between variables.

```
df["review_rating"].describe()
✓ 0.0s

count    3900.000000
mean        3.750051
std         0.713590
min         2.500000
25%         3.100000
50%         3.800000
75%         4.400000
max         5.000000
Name: review_rating, dtype: float64
```

After performing descriptive statistical analysis on the `review_rating` column using the `describe()` function, it can be observed that the **mean and the median values are very close to each other**. This indicates that the distribution of the data is relatively balanced and not strongly affected by extreme values or outliers. When the mean and median are similar, it often suggests that the variable follows an approximately symmetric distribution.

```
df["review_rating"] = df.groupby("category")["review_rating"].transform(lambda x: x.fillna(x.median()))

df.isnull().sum()
✓ 0.0s

customer_id               0
age                       0
gender                    0
item_purchased            0
category                  0
purchase_amount           0
location                  0
size                      0
color                     0
season                    0
review_rating             0
subscription_status       0
shipping_type             0
discount_applied          0
promo_code_used           0
previous_purchases        0
payment_method            0
frequency_of_purchases    0
dtype: int64
```

Based on this observation, missing values in the `review_rating` column can be handled using **either mean imputation or median imputation**, as both methods are generally appropriate when the data does not contain significant outliers. In this analysis, **median imputation was selected** to replace the missing values. The median is often preferred because it is more robust to potential extreme values and provides a stable estimate of the central tendency of the data.

At this stage, the dataset no longer contains **missing values or significant outliers**, as the data cleaning and imputation steps have been successfully completed. Ensuring data quality is an important prerequisite before moving forward with deeper analytical tasks.

4 _ SAMPLE DATA ENGINEERING USING PYTHON :

As a final preprocessing step, **feature engineering** will be performed. Feature engineering involves creating new variables or transforming existing ones in order to extract additional insights from the dataset. This process can improve the analytical value of the data by highlighting patterns or relationships that may not be immediately visible in the original variables.

Applying feature engineering techniques will therefore help enhance the dataset and support a more meaningful and effective analysis in the subsequent stages of the project.

4.1 _ creating new feature age group

so , most feature known for performing on it feature engineering is age

```
df["age"].describe()
```

```
count    3900.000000
mean       44.068462
std        15.207589
min        18.000000
25%        31.000000
50%        44.000000
75%        57.000000
max        70.000000
Name: age, dtype: float64
```

After performing descriptive statistics on the `age` column, it can be observed that the **mean and median values are equal**, indicating a fairly symmetric distribution without extreme skewness. The minimum age is **18**, the maximum age is **70**, and the midpoint (median) is **44**.

To enhance the analysis, a new feature called `age_group` will be created. This feature will **categorize individual ages into groups**, allowing us to analyze patterns and trends across different age ranges more effectively. Grouping ages in this way can provide clearer insights into customer behavior, preferences, or other age-related patterns in the dataset.

```python
def age_category(age):
    if age < 18:
        return "Child"
    elif age < 30:
        return "Young Adult"
    elif age < 45:
        return "Adult"
    elif age < 60:
        return "Senior Adult"
    else:
        return "Elder"

df["age_group"] = df["age"].apply(age_category)

df[["age", "age_group"]].head(10)
```

To create the age_group feature, a custom function named age_category was defined. This function assigns each individual to a specific age group based on their numerical age. The categories are defined as follows:

- **Child:** age less than 18
- **Young Adult:** age between 18 and 29
- **Adult:** age between 30 and 44
- **Senior Adult:** age between 45 and 59
- **Elder:** age 60 and above

The apply() method of the pandas DataFrame was then used to apply this function to each value in the age column. This results in a new column, age_group, which contains the corresponding age category for every individual in the dataset.

	age	age_group
0	55	Senior Adult
1	19	Young Adult
2	50	Senior Adult
3	21	Young Adult
4	45	Senior Adult
5	46	Senior Adult
6	63	Elder
7	27	Young Adult
8	26	Young Adult
9	57	Senior Adult

Displaying the first 10 rows of the age and age_group columns using head(10) allows us to **verify that the categorization has been applied correctly**. This new feature will help in segmenting the dataset and analyzing patterns or trends according to different age groups, which can provide more meaningful insights than using raw age values alone.

4.2 _ creating new feature purchase_frequency_days :

```
df["frequency_of_purchases"].unique()
```

✓ 0.0s

```
array(['Fortnightly', 'Weekly', 'Annually', 'Quarterly', 'Bi-Weekly',
       'Monthly', 'Every 3 Months'], dtype=object)
```

As we can see frequency_of_purchases is an ordinal data , that contains a categorical frequency labels , to make this feature more suitable for analysis we will perform feature transformation by converting categorical frequency labels into numerical values representing the **approximate number of days between purchases**. And put it in our new feature purchase_frequency_days

```
frequency_mapping = {

    "Weekly" : 7,
    "Fortnightly" : 14,
    "Biweekly" : 14,
    "Monthly" : 30,
    "Quarterly" : 90,
    "Every 3 Months" : 90,
    "Annually" : 365
}


df["purchase_frequency_days"] = df["frequency_of_purchases"].map(frequency_mapping)


df[["frequency_of_purchases", "purchase_frequency_days"]].head(10)
```

So I have create a dictionary called frequency_mapping , to map each category to its corresponding number of days:

- "Weekly" → 7 days
- "Fortnightly" / "Biweekly" → 14 days
- "Monthly" → 30 days
- "Quarterly" / "Every 3 Months" → 90 days
- "Annually" → 365 days

The map() function was then applied to the frequency_of_purchases column to create a new column named purchase_frequency_days. This new feature provides a **quantitative representation of purchase frequency**, which is useful for numerical analysis, statistical calculations, and visualizations.

	frequency_of_purchases	purchase_frequency_days
0	Fortnightly	14.0
1	Fortnightly	14.0
2	Weekly	7.0
3	Weekly	7.0
4	Annually	365.0
5	Weekly	7.0
6	Quarterly	90.0
7	Weekly	7.0
8	Annually	365.0
9	Quarterly	90.0

After Displaying the first 10 rows of both frequency_of_purchases and purchase_frequency_days using head(10) allows us to **verify that the mapping has been applied correctly**. By converting categorical frequency data into numerical form, it becomes easier to identify patterns, compare customer behaviors, and perform calculations such as averages or correlations.

4.3 _ features elimination :

As we can see from our dataset we have 2 features discount_applied & promo_code_used

Both of them are binary categorical data , logically we can have discount applied without a promo code has been used ,so I will check for this :

```
df[["discount_applied","promo_code_used"]].head(10)
```

✓ 0.0s

	discount_applied	promo_code_used
0	Yes	Yes
1	Yes	Yes
2	Yes	Yes
3	Yes	Yes
4	Yes	Yes
5	Yes	Yes
6	Yes	Yes
7	Yes	Yes
8	Yes	Yes
9	Yes	Yes

And as you can see my hypothesis was correct , and both of them have redundant information , so to optimize we will remove redundant information.

```
(df["discount_applied"] == df["promo_code_used"]).all()
```

✓ 0.0s

```
np.True_
```

I have made a comparison between the discount_applied and promo_code_used columns using the expression (df["discount_applied"] == df["promo_code_used"]).all(). This check evaluates whether the two columns contain the **exact same values for all rows**. If the result is True, it indicates that promo_code_used does not provide any additional information beyond what is already captured in discount_applied.

```
df=df.drop(columns=["promo_code_used"])

df.columns
```

✓ 0.0s

```
Index(['customer_id', 'age', 'gender', 'item_purchased', 'category',
       'purchase_amount', 'location', 'size', 'color', 'season',
       'review_rating', 'subscription_status', 'shipping_type',
       'discount_applied', 'previous_purchases', 'payment_method',
       'frequency_of_purchases', 'age_group', 'purchase_frequency_days'],
      dtype='object')
```

Since the columns were identical, the promo_code_used column was removed from the dataset using df.drop(columns=["promo_code_used"]). This step simplifies the dataset, reduces redundancy, and ensures that subsequent analysis focuses only on meaningful and unique features.

Finally, df.columns was used to display the updated list of column names, confirming that the redundant column has been successfully removed.

now after we finish data cleaning and feature engineering we will upload our dataset to an RDBMS , and in this project I will use SQL SERVER TO START OUR ANALYSIS .

5_ connecting and uploading dataset to SQL SERVER :

5.1 _ creating SQL SERVER database

So the first step is simple we will sql server and we will create a new database in my case I name it

Customer_shopping_analysis

5.2_ Establishing a Connection to SQL Server

After that we will start the second step which is data integration :

To integrate the cleaned and transformed dataset with a database for further use, we to connect to a **local SQL Server Express instance**. And we will use Python libraries **pyodbc** and **SQLAlchemy** for this purpose.

```python
import pyodbc
from sqlalchemy import create_engine, text

# Local SQL Server Express
server = r".\SQLEXPRESS"
database = "customer_shopping_behavior"

try:
    conn = pyodbc.connect(
        r"DRIVER={ODBC Driver 17 for SQL Server};"
        f"SERVER={server};"
        f"DATABASE={database};"
        "Trusted_Connection=yes;"
    )
    cursor = conn.cursor()
    cursor.execute("SELECT 1")
    print("Connection successful:", cursor.fetchone())
except Exception as e:
    print("Connection failed:", e)
```

The first step in integrating the dataset with a relational database was to **establish a connection to SQL Server Express**. This was done using the Python library **pyodbc**, which allows Python to communicate with SQL Server through the **ODBC protocol**.

The connection parameters specify the **server instance** (.\SQLEXPRESS) and the **database name** (customer_shopping_behavior). The Trusted_Connection=yes option uses Windows authentication for secure access without requiring a username or password.

A try-except block was used to **verify the connection** and handle potential errors. Within the try block, a cursor was created and a simple test query SELECT 1 was executed. If the connection succeeds, the query returns a result and prints "Connection successful". If the connection fails, the except block catches the error and prints "Connection failed" along with the error details.

```python
# 3 Connect to SQL Server
# --------------------------
server = r".\SQLEXPRESS"
database = "customer_shopping_behavior"

# Wrap pyodbc connection in SQLAlchemy engine
conn_str = (
    r"mssql+pyodbc://@.\SQLEXPRESS/customer_shopping_behavior?"
    "driver=ODBC+Driver+17+for+SQL+Server&trusted_connection=yes"
)
engine = create_engine(conn_str)

# --------------------------
# 4 Upload DataFrame to SQL Server
# --------------------------
df.to_sql(
    name="TransformedSuperstore",  # Table name in SQL Server
    con=engine,
    if_exists="replace",           # replace / append / fail
    index=False

)
```

After establishing a database connection, the cleaned and transformed dataset was uploaded to the SQL Server database using **SQLAlchemy**, a Python library that provides a high-level interface for database operations. An engine was created to wrap the connection, enabling seamless communication between Python and SQL Server.

The dataset was written to a new table named TransformedSuperstore using the to_sql() function. The if_exists="replace" option ensures that any existing table with the same name is replaced with the updated dataset, and the DataFrame index was excluded from the table to maintain a clean structure.

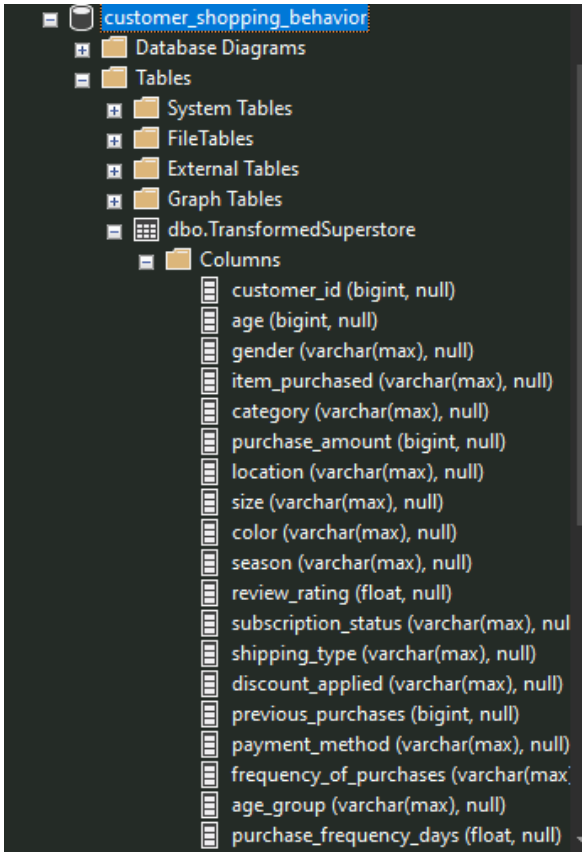
This step allows the dataset to be **stored securely in the database**, making it accessible for future queries, reporting, or integration with other analytical tools. By uploading the DataFrame, the analysis results are preserved and can be shared or reused efficiently.

```python
# --------------------------
# 5 Verify Upload
# --------------------------
with engine.connect() as conn:
    result = conn.execute(text("SELECT TOP 10 * FROM TransformedSuperstore"))
    for row in result:
        print(row)
```

To ensure that the dataset was successfully uploaded to SQL Server, a query was executed to retrieve the first 10 rows from the TransformedSuperstore table. This verification step confirms that the table contains the expected data and that the structure matches the transformed DataFrame.

Performing this check provides confidence that the dataset is **accurately stored and ready for further analysis or reporting**, ensuring the integrity of the database integration process.

And also we will check in our database :



As we can see our cleaned dataset has been successfully uploaded in our database.

6_ data analysis :

I have made 10 question for analysis :

```
--Q1. What is the total revenue generated by male vs. female customers?
--Q2. Which customers used a discount but still spent more than the average purchase
amount?
--Q3. Which are the top 5 products with the highest average review rating?
--Q4. Compare the average Purchase Amounts between Standard and Express Shipping.
--Q5. Do subscribed customers spend more? Compare average spend and total revenue between
subscribers and non-subscribers.
--Q6. Which 5 products have the highest percentage of purchases with discounts applied?
--Q7. Segment customers into New, Returning, and Loyal based on their total number of
previous purchases, and show the count of each segment.
--Q8. What are the top 3 most purchased products within each category?
--Q9. Are customers who are repeat buyers (more than 5 previous purchases) also likely to
subscribe?
--Q10. What is the revenue contribution of each age group?
```

Q1 _ what is the total revenue generated by male vs female customers?

```sql
--Q1 _ what is the total revenue generated by male vs female customers?

select
  gender,
  sum(purchase_amount) as revenue

  from TransformedSuperstore

  group by  gender
```

No issues found

Results | Messages

gender	revenue
Male	157890
Female	75191

The results show that **male customers generated a total revenue of 157,890**, while **female customers contributed 75,191**. This indicates that male customers accounted for a significantly larger portion of the total revenue. These insights can help the business **target marketing efforts or promotions more effectively**, focusing on the segment that drives the most sales.

Q2_ which are the top 5 products with the hightest avreage review rating?

```sql
--Q2 _ which are the top 5 products with the hightest avreage review rating

select top 5
item_purchased,
round(AVG(review_rating),2) as avg_reviews
from TransformedSuperstore
group by item_purchased
order by avg_reviews desc
```

100 % No issues found

Results | Messages

	item_purchased	avg_reviews
1	Gloves	3.86
2	Sandals	3.84
3	Boots	3.82
4	Hat	3.8
5	Handbag	3.78

These findings suggest that these products are generally **well-received by customers**, which could inform **inventory decisions, promotions, and product recommendations**.

Q3_ which customer use a discount but still spent more than the avreage purchase amount?

```sql
--Q3 _ which customer use a discount but still spent more than the avreage purchase amount

select
customer_id,
purchase_amount
from TransformedSuperstore
where
    discount_applied='Yes' and purchase_amount >=(select avg(purchase_amount) from TransformedSuperstore)
```

100 % No issues found Ln: 23, Ch: 9 SPC CRLF

Results Messages

	customer_id	purchase_amount
1	2	64
2	3	73
3	4	90
4	7	85
5	9	97
6	12	68
7	13	72
8	16	81
9	20	90
10	22	62

Q4 _ compare the avreage purchase amounts between standard and express shipping?

```sql
--Q4 _ compare the avreage purchase amounts between standard and express shipping


select
shipping_type,
round(avg(purchase_amount),2) as avg_purshase_amount
from TransformedSuperstore
where shipping_type in ('Standard','Express')
group by shipping_type
order by avg_purshase_amount desc
```

100 % No issues found

Results Messages

	shipping_type	avg_purshase_amount
1	Express	60
2	Standard	58

Q5_ do subscribed customer spend more? compare avreage spend and total revenue between subscribers and non_subscribers?

```sql
--Q5 _ do subscribed customer spend more? compare avreage spend and total revenue between
      --subscribers and non_subscribers
select
subscription_status,
count(customer_id) as total_customer,
round(avg(purchase_amount),2) as avg_spend,
round(sum(purchase_amount),2) as total_revenue
from TransformedSuperstore
group by subscription_status
order by total_revenue,avg_spend desc
```

100 % No issues found Ln: 50

Results Messages

	subscription_status	total_customer	avg_spend	total_revenue
1	Yes	1053	59	62645
2	No	2847	59	170436

Q6_ which 5 products have the highest percentage of purchases with discount applied?

```sql
--Q6 _ which 5 products have the highest percentage of purchases with discount applied?

select top 5
item_purchased,
round(100*sum(case when discount_applied='Yes' then 1 else 0 end)/count(*) ,2 )as discount_rate
from TransformedSuperstore
group by item_purchased
order by discount_rate desc
```

100 % No issues found Ln: 59, Ch: 2

Results | Messages

	item_purchased	discount_rate
1	Hat	50
2	Coat	49
3	Sneakers	49
4	Sweater	48
5	Pants	47

Q7 _ segment customers into new,returning,loyal based on their total number of previous purchases and show the count of each segment?

```sql
-- Q7 _ segment customers into new,returning,loyal  based on their total number of previouse purchases
--and show the count of each segment
with customer_type as(
select
customer_id,
previous_purchases,
case
    when previous_purchases = 1 then 'new'
    when previous_purchases between 2 and 10 then 'returning'
    else 'loyal'
    end as customer_segmented

from TransformedSuperstore
)

select customer_segmented ,
count(*) as number_of_customers
from customer_type
group by customer_segmented
order by number_of_customers desc
```

100 % No issues found Ln: 75, Ch: 2 SPC

Results | Messages

	customer_segmented	number_of_customers
1	loyal	3116
2	returning	701
3	new	83

Q8 _ what are the top 3 most purchased product within each category?

```sql
-- Q8 _ what are the top 3 most purchasedproduct withineach category?
WITH item_counts AS (
    SELECT
        category,
        item_purchased,
        COUNT(customer_id) AS total_orders,
        ROW_NUMBER() OVER (
            PARTITION BY category
            ORDER BY COUNT(customer_id) DESC
        ) AS item_rank
    FROM TransformedSuperstore
    GROUP BY category, item_purchased
)

SELECT
    item_rank,
    category,
    item_purchased,
    total_orders
FROM item_counts
WHERE item_rank <= 3
ORDER BY  item_rank, category;
```

	item_rank	category	item_purchased	total_orders
1	1	Accessories	Jewelry	171
2	1	Clothing	Blouse	171
3	1	Footwear	Sandals	160
4	1	Outerwear	Jacket	163
5	2	Accessories	Belt	161
6	2	Clothing	Pants	171
7	2	Footwear	Shoes	150
8	2	Outerwear	Coat	161
9	3	Accessories	Sunglasses	161
10	3	Clothing	Shirt	169
11	3	Footwear	Sneakers	145

Q9_ are customers who are repeat buyers (more then 5 previouse purchase) also likely to subscribe?

```sql
-- Q9 _ are customers who are repeat buyers (more then 5 previouse purchase) also likely to subscribe?

SELECT
[subscription_status],
COUNT([customer_id]) as repeated_buyers
FROM TransformedSuperstore
WHERE previous_purchases > 5
GROUP BY [subscription_status]
ORDER BY repeated_buyers DESC
```

100 % No issues found Ln: 115, Ch: 30 SPC CRLF

Results | Messages

	subscription_status	repeated_buyers
1	No	2518
2	Yes	958

Q10_ what is the revenue contribution of each age group?

```sql
-- Q10 _ what is the revenue contribution of each age group?

SELECT
age_group,
SUM(purchase_amount) AS total_revnue
FROM TransformedSuperstore
GROUP BY age_group
ORDER BY total_revnue DESC
```

100 % No issues found

Results | Messages

	age_group	total_revnue
1	Senior Adult	68066
2	Adult	65216
3	Young Adult	52905
4	Elder	46894

7 _ sample dashboard :

8_ business recommendations:

Boost subscriptions by promoting exclusive benefit's for subscribers

Customer loyalty program : reward repeated buyers to move them into loyal segment

Review discount policy : balance sales boost with margin control

Product positioning : highlight top-rated and best selling products in campaigns

Targeted marketing : focus efforts on hight revenue age groups and express shipping users